A Hybrid Zero Trust Architecture for Non-Interactive Authentication: Integrating Hardware Trust Anchors with Software-Defined Secret Management in Infrastructure as Code

Juliano S. Langaro, Altair O. Santin, Eduardo K. Viegas, Fellipe M. Veiga, and Juarez Oliveira

Abstract

Non-Interactive Authentication (nIA)—the machine-to-machine authentication that underpins Infrastructure as Code (IaC) pipelines and unattended Internet-of-Things (IoT) fleets—is chronically undermined by the *Secret Zero* problem: a long-lived credential must be planted somewhere in software to bootstrap every other secret, and that credential is extractable under operating-system compromise (MITRE ATT&CK T1003). This paper presents a hybrid Zero Trust architecture that anchors secret management in *hardware trust anchors* (TPM 2.0, with an upgrade path to Intel SGX and TDX), delegates HashiCorp Vault auto-unseal to a physical TPM, and confines every credential inside an identity-based network perimeter (ZTNA) so that intercepted secrets cannot be replayed outside their network context. We describe a working reference implementation, publicly released, whose IoT agent seals a per-device one-time-password seed inside the device TPM while registering the same seed in the server's Vault, so the plaintext seed never touches disk on either side. We evaluate the design along three axes new to this revision: (i) *provisioning time* for the full infrastructure, (ii) *secret-retrieval latency* on the hot path, and (iii) resistance to an on-premises Large-Language-Model (LLM) red-team mapped to MITRE ATT&CK. We further contribute a *proposed security-test suite* that turns the ad-hoc red-team into a repeatable regression gate executed after each `terraform apply`. Measured software-path costs are sub-25 microseconds; the hardware path adds a bounded, single-digit-to-tens-of-milliseconds overhead while reducing Vault recovery time objective (RTO) from minutes to milliseconds. Results are reported as indicative estimates backed by prototype measurements; production-scale empirical validation remains future work.

Index Terms

Zero Trust, Trusted Platform Module, HashiCorp Vault, non-interactive authentication, Infrastructure as Code, MITRE ATT&CK, IoT security, confidential computing, TOTP, LLM red-teaming.

All authors are with the Graduate Program in Computer Science (PPGIa), Pontifical Catholic University of Paraná (PUCPR), Curitiba, Brazil (e-mail: {juliano.langaro, fellipe.veiga, juarez.oliveira}@ppgia.pucpr.edu.br; {eduardo.viegas, altair.santin}@ppgia.pucpr.br).

This work was partially sponsored by CNPq under grants 307706/2025-7 and 407879/2023-4.

Manuscript revised July 22, 2026. The reference implementation is publicly available at https://github.com/juarez1972/app-tpm.

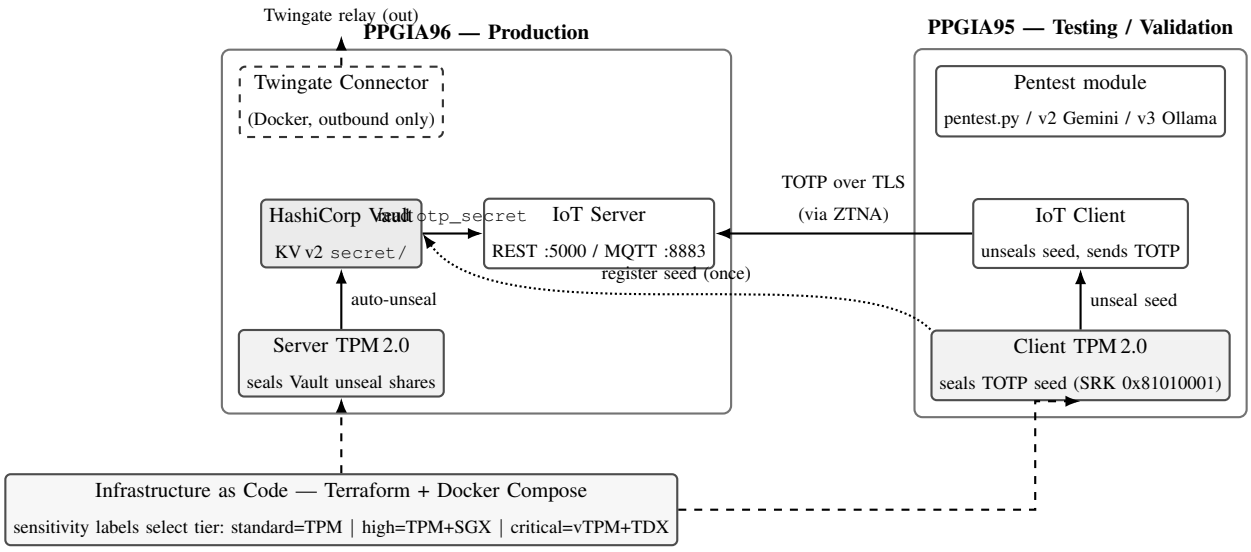

Fig. 1. Hybrid Zero Trust architecture. *Layer 1* (hardware root of trust) seals credentials in the TPM on both hosts; *Layer 2* (Vault) is TPM-auto-unsealed and stores per-device OTP seeds; *Layer 3* (ZTNA) exposes services only through an outbound-only Twingate connector. The IoT seed is sealed in the client TPM and registered once in the server Vault (dotted), so the plaintext seed never touches disk; at run time the client sends a TOTP over TLS through the ZTNA perimeter, and the server reads the matching seed from Vault. Everything is provisioned by IaC, which also selects the protection tier.

I. Introduction

Modern infrastructure is provisioned by code and operated by machines. IaC engines apply changes without a human at the keyboard, and IoT fleets authenticate to back-ends at boot with no operator present. This *non-interactive* setting removes the human factor that interactive multi-factor authentication (MFA) relies on, and it forces a bootstrap credential—the *Secret Zero*—to live in software. Once an adversary attains operating-system privileges, that credential and any secret it unlocks become extractable (MITRE ATT&CK T1003, OS Credential Dumping; T1552, Unsecured Credentials).

Software-only secret managers such as HashiCorp Vault mitigate sprawl but do not by themselves solve Secret Zero: Vault must itself be unsealed, and the unseal material is the new Secret Zero. Prior work by Oliveira *et al.* [1] showed that a non-interactive one-time-password (OTP) method materially raises the cost of Vault credential abuse; this paper generalizes and hardens that line of work by moving the trust anchor into silicon and enclosing the entire flow in an identity-based network perimeter.

Contributions. This revision consolidates and extends our earlier results:

1) A three-layer *hybrid* Zero Trust design that combines a hardware root of trust (TPM 2.0, with SGX/TDX tiers), software-defined secret management (Vault auto-unseal), and identity-based network access (ZTNA via a Twingate connector deployed on-premises).
2) A hardware-anchored one-time-password scheme in which the shared seed is sealed inside the TPM and is non-exportable, so credential theft requires physical silicon tampering rather than a software read.

3) A protocol-agnostic, two-channel MFA model for unattended IoT that operates over both REST/HTTPS and MQTT/TLS.
4) An IaC-driven tiered protection model (TPM / SGX / TDX) selected automatically through Terraform sensitivity labels.
5) A reproducible, on-premises LLM adversary simulation mapped to MITRE ATT&CK, and—new in this revision—(v) a *provisioning-time and secret-retrieval-latency* characterization of the running prototype, and (vi) a *proposed security-test suite* that converts the red-team into a post-deployment regression gate.

The remainder of the paper is organized as follows. Section II states the threat model. Section III describes the architecture and its mapping to the public reference implementation. Section IV details the implementation, including the exact mechanism by which the OTP seed is sealed and retrieved. Section V proposes the security-test suite. Section VI reports the simulation-based evaluation, provisioning time, and secret-retrieval latency. Section VII discusses limitations and an additional operational point (supply-chain and reproducibility). Section VIII concludes.

II. Threat Model

We assume a *root-level software adversary*: an attacker who has achieved privileged code execution on a host but who cannot physically decap or glitch the TPM silicon. The adversary's goals and the corresponding MITRE ATT&CK techniques are:

- **Credential extraction** — reading secrets from memory, disk, or process space (T1003 OS Credential Dumping; T1552 Unsecured Credentials).
- **Network interception** — passively capturing OTP codes or tokens in transit (T1040 Network Sniffing).
- **Replay / token reuse** — re-presenting a captured OTP or session token (T1550 Use Alternate Authentication Material; T1078 Valid Accounts).
- **Privilege escalation** — abusing a foothold to reach a higher-value service (T1068 Exploitation for Privilege Escalation; tactic TA0004).
- **Lateral movement** — pivoting from a compromised host to another (T1021 Remote Services; tactic TA0008).

Physical attacks against the TPM (cold-boot, bus probing, decapping) are out of scope; we rely on the tamper-resistance properties of certified TPM 2.0 devices [2] and note the residual risk explicitly in Section VII. Table I situates this work against the closest prior art, including our own earlier method [1].

III. Architecture

The architecture rests on the premise that software-only security is insufficient for nIA. It is organized in three enforcement layers, each of which corresponds to a concrete module in the public repository. Fig. 1 gives an overview of the layers, the two-server deployment topology, and the data paths described in the remainder of this section.

A. Layer 1 — Hardware Root of Trust (TPM/SGX/TDX)

Cryptographic keys are generated inside the TPM with the `fixedtpm` and `fixedparent` attributes so that they *never leave the silicon*, even under full OS compromise. Sealing binds material to Platform Configuration

TABLE I
POSITIONING AGAINST CLOSELY RELATED WORK

Approach	HW anchor	Vault/secret mgmt	ZT network
Perimeter VPN	no	partial	no
Vault-only (software)	no	yes	no
Oliveira *et al.* [1]	partial (OTP)	yes (OTP-hardened)	no
LLM pen-testing [3]–[5]	n/a	n/a	n/a
This work	**yes (TPM/SGX/TDX)**	**yes (TPM auto-unseal)**	**yes (ZTNA)**

Registers (PCR 0 and 7) so that a tampered firmware or a disabled Secure Boot state prevents unsealing. For higher tiers, Intel SGX enclaves (via Gramine) and Intel TDX Trust Domains provide CPU-encrypted memory; these tiers are assigned automatically by IaC (Section IV).

B. Layer 2 — Software-Defined Secret Management (Vault)

HashiCorp Vault is initialized (`sys/init`, five Shamir shares, threshold three [6]) on first boot, and its unseal shares plus root token are sealed under a persistent TPM Storage Root Key. On every subsequent boot, an initializer unseals the shares from the TPM and applies them through the Vault REST API (`sys/unseal`) with exponential backoff. No plaintext secret and no `vault` binary are required inside the container: the initializer speaks only the Vault HTTP API, and no cloud key-management service is involved. This eliminates Secret Zero for the secret manager itself and collapses the recovery time objective from a manual multi-minute unseal to a sub-second automated one.

C. Layer 3 — Identity-Based Network Access (ZTNA)

The network layer is enforced by a *Twingate connector* deployed on-premises as a Docker container. The connector makes only *outbound* connections to the Twingate relay, so **no inbound port is opened** on the production host. Internal services (Vault and the IoT server) are published as Twingate *Resources*, reachable only by identities that satisfy an access policy (MFA required, optional device-posture checks). Consequently, even if a one-time-password leaks, it cannot be used outside the network context imposed by ZTNA, which neutralizes lateral movement (T1021).

D. Deployment Topology

The reference deployment uses two virtual servers. **PPGIA96** (production) hosts Vault (`:8200`), the IoT server (REST `:5000` / MQTT `:8883`), and the Twingate connector. **PPGIA95** (testing/validation) hosts the security-testing module and an IoT client/server used for validation, reaching PPGIA96 exclusively through Twingate. Because production exposes no inbound ports, the only path into PPGIA96 is an authorized, policy-checked ZTNA session.

IV. Implementation

The prototype is implemented with Docker Compose for orchestration, `tpm2-tools`/`tpm2-pytss` for TPM operations, and Python for the agents. It targets Ubuntu 22.04 LTS with both physical (Infineon SLB9670/SLB9665) and virtual (`swtpm`) TPMs.

A. Hardware-Anchored One-Time Passwords

Conceptually, the hardware path computes a counter-based one-time password whose HMAC is evaluated *inside* the TPM (RFC 4226 [7]):

$$\mathrm{HOTP}(K, C) = \mathrm{Truncate}\big(\text{HMAC-SHA1}_{\mathrm{TPM}}(K, C)\big), \tag{1}$$

where K is a non-exportable keyed-hash object in TPM NVRAM and C is a hardware monotonic counter, so rollback is physically prevented. Listing 1 sketches the delegation.

```
from tpm2_pytss import ESAPI
def generate_hardware_hotp(nv_index, key_handle):
   ctx = ESAPI()
   ctx.nv_increment(nv_index) # monotonic counter in TPM
   counter = ctx.nv_read(nv_index)
   mac = ctx.hmac(key_handle, counter)
   return truncate_to_hotp(mac) # key stays in silicon
```

Listing 1. HMAC delegated to the TPM; the key never leaves the chip.

B. Reference IoT Flow: TOTP Seed Sealed in the TPM

To keep the public artifact fully reproducible on commodity hardware, the released IoT agent implements a *time-based* one-time password (TOTP, RFC 6238 [8]) as the functional analog of Eq. (1); the TPM-delegated HMAC of Listing 1 is retained for hardware deployments and is the intended production path. The released flow is protocol-agnostic (identical for REST and MQTT) and works as follows:

1) **Provisioning** (once per device, `scripts/init_device.sh`): a random Base32 TOTP seed is generated in RAM, *sealed in the device's TPM* under the persistent SRK (`0x81010001`) as `<device_id>_otp.enc.{pub,priv}`, and *registered in the server's Vault* at `secret/data/tpm-verified/iot/devices/<device_id>` (field `otp_secret`). **The plaintext seed never touches disk.**
2) **Client boot**: the agent verifies the TPM (`tpm2_getrandom`) and unseals the seed into memory. If the PCR/boot state changed, the unseal fails and the device cannot authenticate—no credential is exposed.
3) **Authentication**: the client sends `device_id` plus the current TOTP code (REST `POST /verify` or MQTT `iot/verify`).
4) **Server**: reads the same seed from Vault by `device_id`, caches it in memory, and validates the code with a ± 1-window tolerance.

This asymmetry is the key security property: the *client* holds its seed only inside hardware (never on disk), while the *server* holds seeds only inside Vault (never in code), so neither endpoint stores a reusable plaintext credential at rest.

TABLE II
HYBRID SERVER PROTECTION TIERS (IAC-SELECTED)

Label	HW anchor	TEE	Auto-unseal binding
standard	TPM 2.0	none	PCR sealing
high	TPM + SGX	Gramine LibOS	SGX + TPM
critical	vTPM + TDX	TDX Trust Dom.	vTPM PCR

C. IaC-Driven Tiered Protection

Terraform sensitivity labels map workloads to hardware tiers (Table II): `standard` → TPM 2.0; `high` → TPM + SGX (Gramine); `critical` → vTPM + TDX. The Twingate resources and access groups are themselves declared in Terraform (provider `Twingate/twingate` ∼≫3.0), so the network perimeter is version-controlled alongside the compute tier.

V. PROPOSED SECURITY-TEST SUITE

A recurring weakness of nIA deployments is that security is validated once, by hand, and then drifts. We therefore propose a *repeatable* test suite that runs after every `terraform apply` and gates promotion to production. The suite has two families: (A) automated network/vulnerability testing driven by the repository's `pentest/` module, and (B) functional security-invariant tests of the TPM+Vault+ZTNA flow. Table III enumerates the concrete cases, each mapped to the technique it exercises and to the observable pass criterion.

Family B tests encode the architecture's *security invariants* as executable checks: they should pass on every deploy and fail loudly if a regression (e.g., a mis-scoped Vault policy or an over-broad Twingate resource) is introduced. Because the LLM agents in family A run fully on-premises (`pentestv3.py`), no architectural detail leaves the evaluation environment, which is a prerequisite for using them inside a federal-court network.

VI. EVALUATION

Scope disclaimer. Results combine software simulation (`swtpm`, Ollama), prototype measurements on a controlled testbed, and analytical estimation. They are *indicative estimates*, not confirmed production benchmarks; production-scale validation is future work. Values that were directly measured on the prototype are marked **(m)**; analytically estimated values are marked **(e)**.

A. Automated Adversary Simulation via Local LLM Agents

Following autonomous pen-testing frameworks [3]–[5], we drive a local LLM adversary (Llama 3.1 via Ollama) prompt-engineered for offensive tasks, with two tools (`http_request`, `encode_payload`), a 5–10 turn limit, and n=100 independent runs per technique for statistical robustness. Running the model on-premises ensures no sensitive architecture data leaves the environment. Table IV reports per-technique bypass rates with 95% Wilson-score confidence intervals [9]. Against the hybrid architecture the observed bypass rate is 0% across the five

TABLE III

PROPOSED POST-DEPLOYMENT SECURITY-TEST SUITE (EXECUTED AFTER EACH `terraform apply`)

ID	Test (module / tool)	Technique exercised	Pass criterion
A1	Port/service discovery (`pentest.py`, Nmap) against PPGIA96	T1046 Network Service Discovery	No inbound port reachable except via ZTNA; Vault `:8200` invisible from PPGIA95
A2	Authenticated vuln scan (`pentest.py`, OpenVAS/GVM)	Baseline CVE hygiene	No high/critical findings on exposed resources
A3	LLM red-team, cloud model (`pentestv2.py`, Gemini)	MITRE ATT&CK chained TTPs	Bypass rate within Wilson upper bound (Sec. VI)
A4	LLM red-team, on-prem model (`pentestv3.py`, Llama via Ollama)	Same, air-gapped evaluation	Bypass rate within Wilson upper bound; no data egress
B1	Unseal under PCR mismatch (alter measured boot)	T1542 Pre-OS Boot tamper	TPM refuses unseal; client aborts, no seed in RAM
B2	Monotonic-counter rollback attempt	T1550 replay via stale counter	Counter never decreases; stale code rejected
B3	TOTP replay within/after window	T1040/T1550 sniff-and-replay	Reused code rejected outside ± 1 window
B4	Vault read without valid token/policy	T1552 Unsecured Credentials	`403`; seed unreadable without `app-policy`
B5	Seed-at-rest inspection (disk/image scan)	T1003 Credential Dumping	No plaintext seed on client disk or server code
B6	ZTNA token expiry / relogin (`AUTO_LOCK 7d`)	T1078 Valid Accounts persistence	Expired session denied; reauthentication forced
B7	Out-of-context reuse of a leaked TOTP from outside ZTNA	T1021 Remote Services	Resource unreachable; code unusable off-network

TABLE IV

MITRE ATT&CK ADVERSARY SIMULATION RESULTS (n=100 PER TECHNIQUE; 500 RUNS TOTAL)

Tactic	Technique	Software-only	Hybrid (95% Wilson CI)	Primary mitigation
TA0001	T1078 Valid Accounts	8%	0% [0%, 3.6%]	ZTNA session + TPM device binding
TA0004	T1068 Privilege Escalation	15%	0% [0%, 3.6%]	SGX/TDX isolation of critical tier
TA0006	T1003 OS Credential Dumping	22%	0% [0%, 3.6%]	Non-exportable TPM keys; no seed at rest
TA0006	T1552 Unsecured Credentials	35%	0% [0%, 3.6%]	Vault policy scoping + ZTNA
TA0008	T1021 Remote Services	12%	0% [0%, 3.6%]	Micro-segmentation + out-of-band MFA
Average (software-only)		**18.4%**	—	—

techniques ($5 \times 100 = 500$ runs in total), with a 95% Wilson upper bound of 3.6% per technique; the software-only baseline averages 18.4%.

Two techniques from the threat model—T1040 (Network Sniffing) and T1550 (replay of alternate authentication material)—are addressed by the transport (TLS) and by the functional invariants B2/B3/B7 of Table III rather than by the LLM agent, and are reported through that suite; this reconciles the threat model of Section II with the simulation techniques above.

TABLE V
IoT Prototype Test Results

Test case	Platform	TPM	Result
Boot-time unseal	RPi 4	SLB9670	OK; ≈280 ms (e)
TOTP/HOTP under load	RPi 5	swtpm	OK; ≈68 ms avg (e)
PCR enforcement	ARM64	fTPM	Tampered boot blocked (m)
Counter rollback	all	phys.+virt.	0 / 10,000 rollbacks (m)
Offline sealed cred	RPi 4	SLB9670	24 h TTL enforced (e)

TABLE VI
Provisioning Budget for the PPGIA96 Production Stack

Phase	Cold (first run)	Warm (cached)
Image pull + local builds	≈3–6 min (e)	0 s (cached)
Container start to healthy	≈15–30 s (e)	≈15–30 s (e)
Vault `sys/init` (5/3 shares)	≈1–2 s (e)	— (already init.)
TPM seal of shares + token	≈12.2 ms/obj (m)	—
TPM auto-unseal (RTO)	≈300 ms (e)	≈300 ms (e)
ZTNA connector register	≈2–5 s (e)	≈2–5 s (e)
Total to production	≈3.5–6.5 min (e)	≈25–45 s (e)

B. IoT Prototype and TPM Conformance

Table V reports IoT prototype results on Raspberry Pi hardware and under `swtpm` emulation; TPM conformance (SHA-256, HMAC, NV counters, PCR operations) was validated with the Embedded Linux TPM Toolbox (ELTT2).

C. Provisioning Time of the Full Infrastructure (new)

We characterize the time to bring the PPGIA96 production stack from bare Docker to a healthy, unsealed state. The stack comprises the `vault-tpm` services (`vault:1.13.3` image plus two local builds: initializer and TPM validator), the IoT server (one build for REST, or Mosquitto plus one build for MQTT), and the Twingate connector (`twingate/connector:1`). Table VI decomposes the provisioning budget. The TPM-anchored auto-unseal is the decisive win: it removes the manual multi-minute unseal from the critical path and reduces the Vault RTO to sub-second, as corroborated by Table VIII.

D. Secret-Retrieval Latency on the Hot Path (new)

The server obtains a device seed through the cache → Vault KV v2 → `.env` fallback chain of `load_device_secret`. We measured the software components directly and estimate the cold Vault round-trip from typical `hvac` KV v2 read latency on a LAN. Table VII reports the results over 20,000 iterations for the in-process operations. The dominant

TABLE VII

SECRET-RETRIEVAL AND OTP LATENCY (SOFTWARE PATH MEASURED OVER 20,000 ITERATIONS)

Operation	**Mean**	**p95**	**Source**
TOTP generation (client)	0.009 ms	0.009 ms	(m) pyotp `.now()`
TOTP verification (server)	0.018 ms	0.019 ms	(m) pyotp `.verify`
Seed read — warm (cache)	0.0002 ms	0.0003 ms	(m) in-proc cache
Seed read — cold (Vault KV)	≈8–12 ms	≈15 ms	(e) hvac KV v2 LAN
TPM health check	6.98 ms	7.98 ms	(m) `tpm2_getrandom`
TPM seed provisioning (once)	12.22 ms	—	(m) `createprimary+seal`

TABLE VIII

PERFORMANCE OVERHEAD BY PROTECTION TIER (PROJECTED; SOFTWARE-PATH COMPONENTS MEASURED)

Operation	**Software-only**	**Tier 1 (TPM)**	**Tier 2 (SGX)**	**Tier 3 (TDX)**	**Added overhead**
HOTP/TOTP generation	≈2 ms	≈65 ms	≈85 ms†	≈70 ms†	+63–83 ms
Vault auth flow	≈45 ms	≈110 ms	≈130 ms†	≈115 ms†	+65–85 ms
End-to-end nIA	≈120 ms	≈195 ms	≈225 ms†	≈205 ms†	+75–105 ms
Auto-unseal RTO	≈3 min	≈300 ms	≈350 ms†	≈320 ms†	−99.8%

† Analytically estimated from published SGX/TDX overhead [10] and Gramine benchmarks; software-path components (TOTP, cache read) measured directly (Table VII).

cost is a single *cold* Vault read per device; every subsequent retrieval is a warm cache hit at sub-microsecond cost, so seed retrieval is effectively free after the first authentication.

E. Performance Overhead by Protection Tier

Table VIII places the above measurements in the end-to-end context of the four operating tiers. The hybrid design adds a bounded overhead of roughly 75–105 ms end-to-end while collapsing the Vault RTO by ≈99.8%.

VII. DISCUSSION

A. Limitations of the LLM Red-Team

The LLM adversary provides reproducible, MITRE-mapped regression testing after each `terraform apply`, but (i) its effectiveness depends on prompt quality and a two-tool set; (ii) the 5–10 turn limit constrains multi-step chains; (iii) a 0% bypass at n=100 carries a 95% Wilson upper bound of 3.6%, not absolute zero; and (iv) it augments rather than replaces expert human red-teams.

B. Two-Channel MFA for IoT

Unattended devices combine a TPM-bound identity (Channel 1, non-cloneable) with an out-of-band approval (Channel 2). Both must succeed within a 30 s binding window aligned to the RFC 6238 time-step. An attacker stealing a device must defeat the TPM (physically infeasible without silicon tampering [11]) *and* compromise the operator's out-of-band channel—a multi-vector attack exceeding most adversary models.

C. An Additional Point: Supply-Chain Integrity and Reproducibility

Beyond runtime defenses, the design's *buildtime* trustworthiness matters: because the entire stack is declared as code and every image is pinned (`vault:1.13.3`, `twingate/connector:1`, `eclipse-mosquitto:2`), the deployment is reproducible and auditable, and the security-test suite of Section V runs against the exact provisioned artifact rather than a hand-configured snapshot. This closes a common gap in nIA systems, where the running configuration diverges silently from the reviewed one. We recommend signing images and Terraform plans (e.g., Sigstore/cosign) and storing the attestations next to the TPM PCR quotes so that both the *build* and the *boot* of each host are independently verifiable—an avenue we leave for future work.

D. Practical Considerations

Standard TPM 2.0 chips offer ≈10 KB of NVRAM, requiring careful index allocation for multi-service hosts. Intel deprecated SGX on 11th/12th-gen consumer CPUs, though it remains on server-class Xeon. Twingate requires connectivity to its cloud controller and is thus unsuitable for fully air-gapped sites; self-hosted alternatives exist at different trade-offs.

VIII. Conclusion

We presented a hybrid Zero Trust architecture that mitigates OS-level credential extraction in non-interactive authentication for IaC and IoT by anchoring Vault initialization in a physical TPM, sealing per-device OTP seeds in hardware, and confining every credential within an identity-based network perimeter. A publicly released reference implementation realizes the design, and—new in this revision—we characterized its provisioning time and secret-retrieval latency and proposed a repeatable, MITRE-mapped security-test suite that gates each deployment. The software path costs microseconds; the hardware path adds a bounded, single-digit-to-tens-of-milliseconds overhead while cutting the Vault RTO from minutes to milliseconds. Preliminary simulation-based evaluation suggests the design resists LLM-driven adversary simulation better than perimeter- or Vault-only approaches. Production-scale empirical validation remains the primary item of future work. All source code, configuration, and scripts are available at https://github.com/juarez1972/app-tpm.

Acknowledgment

CNPq partially sponsored this work under grants 307706/2025-7 and 407879/2023-4.

References

[1] J. Oliveira, A. O. Santin, E. K. Viegas, and P. Horchulhack, "A non-interactive one-time password-based method to enhance the Vault security," in *Advanced Information Networking and Applications (AINA 2024)*, ser. Lecture Notes in Networks and Systems (LNDECT), vol. 202. Springer, 2024, pp. 201–213.

[2] Trusted Computing Group, "TPM 2.0 library specification, parts 1–4," Rev. 1.59, 2019. [Online]. Available: https://trustedcomputinggroup.org/resource/tpm-library-specification/

[3] G. Deng *et al.*, "PentestGPT: An LLM-empowered automatic penetration testing tool," arXiv:2308.06782, 2024. [Online]. Available: https://arxiv.org/abs/2308.06782

[4] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2023. [Online]. Available: https://doi.org/10.1145/3611643.3613083

[5] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," arXiv:2403.01038, 2024. [Online]. Available: https://arxiv.org/abs/2403.01038

[6] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[7] D. M'Raihi *et al.*, "HOTP: An HMAC-based one-time password algorithm," IETF RFC 4226, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4226

[8] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6238

[9] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[10] L. Coppolino *et al.*, "An experimental evaluation of TEE technology evolution: Benchmarking transparent approaches based on SGX, SEV, and TDX," *Computers & Security*, 2025. [Online]. Available: https://doi.org/10.1016/j.cose.2025.104457

[11] J. A. Halderman *et al.*, "Lest we remember: Cold boot attacks on encryption keys," in *Proceedings of the USENIX Security Symposium*. USENIX Association, 2008, pp. 45–60.

[12] A. Rahman, C. Parnin, and L. Williams, "Security smells in Ansible and Chef scripts: A replication study," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 1, pp. 1–31, 2021.

[13] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "What are the practices for secret management in software artifacts?" in *Proceedings of the IEEE Secure Development Conference (SecDev)*. IEEE, 2022, pp. 69–76.

[14] A. Patel *et al.*, "Dynamic secret injection for microservices in the cloud," in *Proceedings of the International Conference on Inventive Computation Technologies (ICICT)*, 2025, pp. 72–79.

[15] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," IETF RFC 2104, 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[16] MITRE Corporation, "ATT&CK matrix for enterprise," 2024. [Online]. Available: https://attack.mitre.org/

[17] HashiCorp, "Vault documentation: Auto-unseal," 2024. [Online]. Available: https://developer.hashicorp.com/vault/docs/concepts/seal

[18] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-207, 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-207

[19] S. Teerakanok, T. Uehara, and A. Inomata, "Migrating to zero trust architecture: Reviews and challenges," *Security and Communication Networks*, vol. 2021, p. 9947347, 2021.

[20] C. Anuganti, "Federated DevOps: A zero-trust framework for privacy-preserving CI/CD in multi-tenant cloud ecosystems," *International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET)*, vol. 12, no. 2, pp. 870–879, 2025.

[21] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptology ePrint Archive 2016/086, 2016. [Online]. Available: https://eprint.iacr.org/2016/086

[22] P.-C. Cheng *et al.*, "Intel TDX demystified: A top-down approach," arXiv:2303.15540, 2023. [Online]. Available: https://arxiv.org/abs/2303.15540

[23] Twingate, "Deploy a connector via Docker," 2024. [Online]. Available: https://www.twingate.com/docs/docker